CalderR.

RETHINKING · HOW · WORK · WORKS

Agentic AI Engineering Internship , 2026

WEEK 5

AI Fundamentals & Agentic AI Foundations

Monday 20 July – Friday 24 July 2026

Orchestrating teams of AI agents to solve complex problems collaboratively

Week Overview

Week 5 is where agentic AI engineering becomes genuinely powerful. Interns move from single-agent systems to coordinated teams building orchestrators that delegate, specialists that focus, and arbiters that resolve disagreement. By Friday, every intern will have a multi-agent system running in production that no single-agent approach could replicate.

Detail	Information
Week Dates	Monday 20 July – Friday 24 July 2026
Theme	Multi-Agent Systems
Primary Stack	Python · LangGraph · LangChain · Groq API · FastAPI · SQLite
Total Commitment	20 hours (4 hours/day · 5 days)
Deliverable Due Friday	1 Intermediate Project + 1 Production Project (choose one of each)

Learning Objectives

- Understand multi-agent architectures: orchestrator-worker, supervisor, peer-to-peer, hierarchical
- Build structured agent-to-agent communication with typed message passing
- Implement supervisor agents that delegate to specialist agents and aggregate results
- Handle inter-agent failures, timeouts, conflicts, and disagreements gracefully
- Design agent personas with distinct specializations, tools, and authority scopes
- Instrument a multi-agent system with per-agent observability (decisions, timing, cost)
- Build a complete production multi-agent system with 4 or more specialized agents

Topics & Subtopics

Topic	Subtopics
Multi-Agent Architectures	Orchestrator-worker, supervisor pattern, peer networks, hierarchical teams, pipeline agents, fan-out/fan-in
Agent Communication	Message passing, shared state, typed handoff messages, result schemas, cross-agent context windows
Specialization Design	Agent personas, domain expertise prompting, tool assignment per agent, capability declaration, authority scopes
Conflict & Consensus	Voting mechanisms, confidence-weighted consensus, arbitration agents, majority vs unanimous agreement
Failure Handling	Agent timeouts, retry with alternate agent, partial-result recovery, graceful degradation, circuit breakers
LangGraph Multi-Agent	Agent nodes, supervisor chains, cross-agent state, subgraph delegation, parallel fan-out in LangGraph
Observability	Per-agent logging (token usage, latency, decisions), LangSmith traces, agent audit trails, cost attribution

Daily Breakdown

Each day builds on the last. By Wednesday you will have a working hierarchical team. Thursday introduces debate and consensus patterns. Friday is integration, polish, and presentation.

Day	Focus	Core Learning	Applied Practice
Mon	**Foundations & First Team**	Study CAMEL, AutoGen, and AgentVerse papers. Map architectures: orchestrator-worker vs supervisor vs peer network. Identify the decision that separates them.	Build your first 2-agent system: Research Agent + Synthesis Agent. Researcher queries multiple sources (mock tools), Synthesis Agent produces a structured report. Type all inter-agent messages with Pydantic.
Tue	**Supervisor Pattern**	Study LangGraph supervisor pattern in depth. Build a Supervisor Agent that receives a complex task, decomposes it into subtasks, delegates to 3 specialist agents, and aggregates typed results.	Add failure resilience: if a specialist times out or returns low-confidence output, supervisor reroutes to a fallback agent. Log every delegation decision with reasoning.
Wed	**Hierarchical Teams**	Build a 2-level hierarchy: Executive Supervisor → [Research Lead, Engineering Lead] → 4 Worker Agents. Study how state flows across hierarchy levels without leaking context.	Build: Full software delivery simulation. PM Agent decomposes requirements → Backend Agent + Frontend Agent build in parallel → QA Agent tests → PM Agent synthesises a release report.
Thu	**Debate & Consensus**	Study debate pattern and confidence-weighted consensus. Build a 3-agent debate: Proposer, Challenger, Arbiter. Arbiter weighs argument quality, not recency.	Build: Multi-perspective code reviewer. Security Agent, Performance Agent, and Maintainability Agent each review the same code independently. Consensus Agent synthesises a final

			verdict with conflict annotations.
Fri	**Production Integration**	Add full observability to your multi-agent system: per-agent token counts, latency, decision logs, cost attribution. Expose a structured audit trail via FastAPI.	Standup: demo with live agent activity visualised in terminal or Streamlit. Architecture review. Answer: when does one agent outperform a team, and when does it fail?

Recommended Resources

Read the papers before building. The architectural intuitions from CAMEL and AutoGen will save you hours of trial and error.

Type	Resource	Why It Matters
Paper	CAMEL: Communicative Agents for Mind Exploration (2023)	Foundational defines role-playing multi-agent communication
Paper	AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (Microsoft, 2023)	Production multi-agent framework from Microsoft Research
Paper	AgentVerse: Facilitating Multi-Agent Collaboration (2023)	Structured collaboration patterns and conflict resolution
Paper	Scalable Multi-Agent Reinforcement Learning (2024 survey)	How agents learn to coordinate background context
Docs	LangGraph Multi-Agent Supervisor Tutorial (official)	Canonical LangGraph multi-agent implementation
Docs	LangGraph Subgraphs Documentation	Delegation via nested graphs essential for hierarchical teams
Video	Multi-Agent Systems with LangGraph LangChain YouTube	Walkthrough of supervisor and worker patterns
Blog	Andrew Ng AI Agentic Design Patterns (2024)	High-level taxonomy of agent patterns from a trusted source
Repo	AutoGen Examples Microsoft GitHub	Production multi-agent implementations with real use cases
Blog	Building Production Multi-Agent Systems: Lessons Learned (2025)	Real-world engineering lessons and failure modes

Hands-on Labs

All three labs must be completed before starting your projects. Each one targets a distinct architecture pattern from the week.

Lab 5.1 Typed Message Bus

Build a message-passing backbone for multi-agent systems. Define 4 typed Pedantic message schemas (TaskRequest, TaskResult, ErrorReport, Handoff). Implement a simple in-memory message bus. Build 3 agents that communicate exclusively through typed messages no raw strings. Verify that a malformed message raises a validation error before any agent receives it.

Goal Develop the discipline of typed inter-agent communication the foundation of every reliable multi-agent system.

Lab 5.2 Supervisor with Failure Recovery

Build a Supervisor Agent that delegates to 3 specialist agents. Inject failures into 2 specialists (random timeout, low-confidence response). The supervisor must: detect the failure type, log its reasoning for re-routing, try an alternative agent, and if all alternatives fail produce a gracefully degraded response. The system must never crash.

Goal Understand that agent failure is not exceptional it is routine. Resilient supervisor logic is non-negotiable in production.

Lab 5.3 Consensus Engine

Build a 4-agent consensus system: 3 specialist agents each produce a structured opinion (answer + confidence score 0–1 + reasoning). A Consensus Agent aggregates opinions using confidence-weighted voting. If no option clears 60% weighted confidence, Consensus Agent requests a second round from only the top 2 agents. Output the final answer with a dissent summary if agents disagreed.

Goal Consensus with explicit confidence and dissent tracking produces trustworthy outputs. Learn when to surface disagreement rather than hide it.

Weekly Assessment

Answer all six questions in writing before Friday's standup. Depth matters more than length.

Type	Question
Conceptual	Explain the difference between a supervisor pattern and a peer network. What problem does each solve best?
Conceptual	How does typed message passing between agents improve system reliability compared to passing raw strings or dicts?
Conceptual	When does splitting a task across multiple agents hurt rather than help? Give two concrete failure scenarios.
Technical	Write the Pydantic schema for a Handoff message between a Research Agent and a Synthesis Agent. What fields are essential?
Technical	How would you detect and handle a situation where two agents in a consensus system produce directly contradictory outputs with equal confidence?
Design	You are building a multi-agent customer support system. Design the agent hierarchy: who talks to whom, what each agent knows, and what happens when the top-level agent is unavailable.

Weekly Standup Requirements

Prepare each item before Friday. The demo is live no slides, no recordings.

Item	Expectation
Live Demo	Show at least 4 agents running in sequence or parallel. Narrate what each agent decides and why.
Agent Decision Log	Show a printed or rendered log of every agent's key decision during the demo run.
Architecture Review	Draw the full agent graph: nodes (agents), edges (message flow), and failure paths. Explain every design choice.
Code Review	Share your typed message schema and supervisor routing logic. Defend the design.
Failure Test	Deliberately break one agent during the demo. Show how the system degrades gracefully.
Reflection	One honest answer to: what did you get wrong in your initial design, and what did you change?

Week 5 Projects

This week's projects are intentionally ambitious. They are designed to appear on your GitHub, your LinkedIn, and in interviews. Every project requires a real architecture decision there is no scaffolding. Choose one Intermediate and one Production project.

Portfolio Standard Every project must ship with: an architecture diagram, a written system design doc (README), a recorded or live demo, and meaningful test coverage. Anything less is not portfolio-ready.

Category 1 Projects Choose One

Intermediate projects should take 7–10 hours. They are scoped to demonstrate mastery of a specific multi-agent pattern at production code quality.

Project 5-I-A: Autonomous Competitive Intelligence Agent

Autonomous Competitive Intelligence Agent	
Difficulty	Intermediate
Overview	Build a multi-agent system that takes a company name, autonomously researches it from multiple angles (market position, product, technology stack, recent news, financials), and produces a structured competitive intelligence briefing the kind a strategy team would pay for.
Problem Solved	Manual competitive research is time-consuming, inconsistent, and shallow. A team of specialized AI agents can gather, cross-reference, and synthesize information an order of magnitude faster with a consistent output schema.
System Architecture	Orchestrator Agent → parallel fan-out → [Market Agent, Product Agent, Tech Stack Agent, News Agent, Sentiment Agent] → Synthesis Agent → Conflict Resolver (if agents contradict) → Report Generator → FastAPI endpoint → Streamlit display

Agent Roles	Orchestrator: plans research strategy and assigns sub-questions. Market Agent: analyses market position and sizing. Product Agent: maps features and differentiators. Tech Stack Agent: infers technology choices. News Agent: surfaces recent developments. Sentiment Agent: gauges public/analyst sentiment. Synthesis Agent: merges findings with confidence scores. Conflict Resolver: flags and adjudicates contradictions between agents.
Technical Skills	LangGraph fan-out/fan-in, parallel agent execution, typed Pydantic reports, confidence scoring, conflict detection, FastAPI, Streamlit
Portfolio Deliverables	GitHub repo with full README and system design doc, FastAPI endpoint accepting company name, Streamlit UI showing agent activity in real time, 3 sample intelligence reports on real public companies, architecture diagram, recorded 3-minute demo video
What Makes It Stand Out	Real parallel agent execution (not sequential). Conflict resolution with explicit reasoning. Structured output that looks professional enough to share with a non-technical stakeholder. The architecture diagram alone demonstrates senior-level systems thinking.
Evaluation Criteria	All 5 research agents run and produce typed outputs. Synthesis is coherent and non-redundant. Conflicts are detected and surfaced explicitly. FastAPI returns a valid JSON report. Streamlit shows live agent status. README includes a full system design section.

Project 5-I-B: Multi-Agent Legal Document Reviewer

Multi-Agent Legal Document Reviewer	
Difficulty	Intermediate
Overview	Build a legal document analysis system where specialist agents review contracts from different legal perspectives and a Judge Agent produces a final risk assessment. The system identifies risky clauses, missing protections, and ambiguous language and explains its reasoning.
Problem Solved	Legal review is expensive, slow, and often inconsistent between reviewers. An AI team of specialized legal reviewers can surface risk in minutes and because each agent has a distinct mandate, the output is more thorough than any single reviewer.

System Architecture	Document Ingestion → [Risk Agent, Compliance Agent, Liability Agent, Obligations Agent] (parallel review) → Debate Round (agents challenge each other's findings) → Judge Agent (final assessment with confidence + dissent log) → Markdown Report
Agent Roles	Risk Agent: identifies unfavorable terms and missing protections. Compliance Agent: checks for regulatory red flags. Liability Agent: maps liability exposure and indemnification gaps. Obligations Agent: extracts all party obligations and deadlines. Debate Facilitator: runs one structured challenge round. Judge Agent: weighs arguments, assigns severity scores (1–5) per clause, and produces the final report.
Technical Skills	Debate pattern, confidence-weighted consensus, clause extraction, severity scoring, LangGraph state sharing, Markdown report generation
Portfolio Deliverables	CLI + Streamlit app, 3 sample contracts analyzed (use open-source templates), per-clause risk report with severity scores and agent attribution, debate transcript log, system design README, architecture diagram
What Makes It Stand Out	The debate round produces a qualitatively better output than a single-pass review. The per-clause agent attribution lets users understand *why* a clause was flagged, not just *that* it was. This is a demo that impresses non-technical interviewers.
Evaluation Criteria	All 4 review agents process the same document in parallel. Debate round produces at least one changed finding. Judge Agent report includes severity scores and dissent notes. Streamlit shows clause-level annotations. Three complete sample reports committed to repo.

Project 5-I-C: AI Product Manager Feature Prioritisation Engine

AI Product Manager Feature Prioritisation Engine	
Difficulty	Intermediate
Overview	Build a product prioritisation system where 4 agents representing different stakeholder perspectives (engineering, customer success, revenue, strategy) evaluate a backlog of features and vote on priority. A PM Agent facilitates, challenges weak reasoning, and produces a final prioritised roadmap with documented trade-offs.

Problem Solved	Feature prioritisation is one of the highest-stakes decisions in a product org and it is routinely done poorly driven by whoever speaks loudest. A structured multi-agent system enforces discipline: every agent must justify its vote, and every trade-off is documented.
System Architecture	Feature Backlog Input (JSON) → PM Agent (orchestrates) → Parallel: [Engineering Agent, Customer Success Agent, Revenue Agent, Strategy Agent] (each scores features + justification) → Debate Round (PM Agent challenges scores below threshold) → Ranking Aggregator → Roadmap Generator → Streamlit UI
Agent Roles	Engineering Agent: scores based on technical complexity, debt, and risk. Customer Success Agent: scores based on customer pain and support volume. Revenue Agent: scores based on ARR impact and upsell potential. Strategy Agent: scores based on competitive differentiation and vision alignment. PM Agent: facilitates, challenges outlier scores, and synthesises the roadmap. Ranking Aggregator: applies weighted voting with configurable stakeholder weights.
Technical Skills	Weighted voting, structured reasoning schemas, debate facilitation, configurable agent weights, roadmap generation, Streamlit interactive UI
Portfolio Deliverables	Streamlit app with feature input form, configurable stakeholder weights via sliders, live agent scoring display, final roadmap with trade-off documentation, 2 worked examples with commentary, architecture diagram, README
What Makes It Stand Out	Configurable stakeholder weights make this a tool, not just a demo. The debate round where PM Agent challenges outlier scores produces more defensible outputs. The trade-off documentation is the kind of artefact real PMs generate before board meetings.
Evaluation Criteria	All 4 stakeholder agents score independently with typed output. PM Agent challenges at least one outlier score. Final roadmap is correctly ranked by weighted score. Stakeholder weights are configurable in UI without code changes. Two complete worked examples in README.

Category 2 Projects Choose One

Production projects should take 10–15 hours across this week and next. They are portfolio centrepieces deployed, documented, and demo-ready for technical interviews.

Hiring Signal A well-executed production project from this week alone is sufficient to demonstrate junior AI engineering capability to most companies hiring in 2026.

Project 5-P-A: Autonomous AI Research Lab

Autonomous AI Research Lab	
Difficulty	High
Overview	Build a fully autonomous research system that accepts a research question, assembles a dynamic team of specialist agents based on the question's domain, executes a structured multi-phase research process (hypothesis generation, evidence gathering, analysis, peer review), and publishes a professional research report all without human intervention.
Problem Solved	Deep research on a new topic requires hours of reading, synthesis, and critical evaluation. Organisations that could automate this process gain a compounding intelligence advantage. This project proves you can build that infrastructure.
System Architecture	Research Question Input → Domain Classifier → Dynamic Agent Assembler (selects 3–5 specialist agents based on domain) → Phase 1: Hypothesis Generator Agent → Phase 2: Evidence Agents (parallel, each assigned a sub-question) → Phase 3: Critic Agent (challenges evidence and hypotheses) → Phase 4: Synthesis Agent → Phase 5: Peer Review Agent (second-pass quality check) → Report Publisher → FastAPI REST API → Streamlit UI with phase-by-phase progress → Docker Compose deployment
Agent Roles	Domain Classifier: identifies the research domain and selects appropriate specialist agents. Evidence Agents (dynamic, 3–5): each assigned a specific sub-question, uses RAG over a seeded document store + tool calls. Hypothesis Generator: proposes a structured hypothesis before evidence is gathered. Critic Agent: challenges evidence quality and hypothesis alignment explicitly marks weak links. Synthesis Agent: writes the report with evidence citations. Peer Review Agent: checks the report for internal contradictions and

	unsupported claims. Report Publisher: formats and publishes to FastAPI endpoint.
Technical Skills	Dynamic agent assembly, hierarchical multi-phase orchestration, RAG integration within agents, critic pattern, peer review pattern, LangGraph subgraphs, FastAPI, Docker, Streamlit, Pydantic report schemas
Portfolio Deliverables	GitHub repo (public, portfolio-ready) with full system design README, Docker Compose deployment, FastAPI API with OpenAPI docs, Streamlit UI showing phase-by-phase agent progress, 5 complete research reports on different domains, LangSmith trace exports, architecture diagram (draw.io or Excalidraw), recorded 5-minute demo video, a published blog post (dev.to or Hashnode) explaining the system design
What Makes It Stand Out	Dynamic agent assembly means the system adapts to the question not every system does this. The Critic Agent produces demonstrably better outputs and is a pattern rarely seen in intern portfolios. The published blog post turns a project into an artefact that recruiters and hiring managers discover independently.
Evaluation Criteria	Dynamic agent selection works across at least 3 different domains. All 5 research phases execute and produce typed outputs. Critic Agent modifies at least one finding in 3 out of 5 test runs. Docker Compose starts with one command. FastAPI returns a valid JSON report. Streamlit shows real-time phase progress. Five published reports committed to repo. Blog post live and linked from README.

Project 5-P-B: Multi-Agent Software Engineering Platform

Multi-Agent Software Engineering Platform	
Difficulty	High
Overview	Build an end-to-end AI software team that takes a natural-language feature specification and autonomously produces a working, tested, containerised application. The team includes a PM, Architect, Backend Engineer, Frontend Engineer, QA Engineer, and Security Reviewer each with distinct tools and authority. The output is a runnable Docker container with a passing test suite.

Problem Solved	The bottleneck in software delivery is not ideas it is execution bandwidth. An AI engineering team that can take a spec to a running app in minutes represents a genuine productivity multiplier. Building this system proves you understand both AI engineering and software engineering.
System Architecture	Spec Input (natural language) → PM Agent (decomposes into tasks + acceptance criteria) → Architect Agent (designs system, writes component contracts) → Parallel: [Backend Agent (Python/FastAPI), Frontend Agent (HTML/JS)] → Integration Agent (wires components together) → QA Agent (writes + executes pytest suite) → Security Agent (static analysis, flags issues) → DevOps Agent (writes Dockerfile + docker-compose) → Validation Agent (runs docker build, confirms tests pass) → GitHub Agent (creates repo, commits code, opens PR) → FastAPI Control Plane → Streamlit Project Dashboard
Agent Roles	PM Agent: natural language → structured task list with acceptance criteria. Architect Agent: designs component interfaces as typed contracts (Pydantic models). Backend Agent: writes FastAPI application code matching the contracts. Frontend Agent: writes HTML/CSS/JS consuming the API. QA Agent: writes pytest tests against acceptance criteria; executes them in a subprocess sandbox. Security Agent: scans generated code for OWASP Top 10 patterns; flags critical issues for revision. DevOps Agent: writes Dockerfile and docker-compose.yml; verifies the image builds. Validation Agent: runs the full stack, hits the API, confirms acceptance criteria pass. GitHub Agent: creates GitHub repo via API, commits all artifacts, opens PR.
Technical Skills	Code generation at scale, sandboxed code execution, test generation and execution, static security analysis, Docker generation, GitHub API, LangGraph complex state, full-stack generation, multi-agent feedback loops
Portfolio Deliverables	GitHub repo with full system design README, 3 generated and deployed mini-applications (each in its own GitHub repo with passing CI), Streamlit dashboard showing agent activity and code generation in real time, security scan reports for each generated app, architecture diagram, recorded 5-minute demo, blog post explaining the architecture
What Makes It Stand Out	A system that generates working software end-to-end is a genuinely rare portfolio piece. The Security Agent and GitHub Agent push this well beyond typical code-generation projects. Three generated repos in GitHub each with passing tests is concrete, verifiable proof of capability.

Evaluation Criteria	PM Agent produces structured acceptance criteria. Architect Agent produces typed component contracts that downstream agents follow. Generated backend and frontend integrate without manual fixes. QA Agent tests pass in at least 2 of 3 generated apps. Security Agent flags at least one real issue per app. Docker image builds on first attempt in at least 2 of 3 apps. GitHub repos are public and linked in README.

Project 5-P-C: Real-Time Multi-Agent Market Intelligence System

Real-Time Multi-Agent Market Intelligence System	
Difficulty	High
Overview	Build a live market intelligence platform where a team of agents continuously monitors configurable topics (companies, technologies, trends), each producing structured signals. A meta-agent synthesises signals across agents, detects emerging patterns, and alerts when something important is happening all surfaced in a real-time dashboard.
Problem Solved	Intelligence teams at companies, funds, and consultancies spend enormous resources manually monitoring markets. An always-on multi-agent system that continuously synthesises signals and surfaces anomalies is a genuine enterprise product and a standout portfolio demonstration.
System Architecture	Config (topics, agents, intervals) → Agent Scheduler (triggers agents on interval or event) → Parallel Specialist Agents [Company Monitor Agent, Technology Signal Agent, Competitor Activity Agent, Sentiment Agent, Regulatory Agent] → Signal Store (SQLite with time-series schema) → Pattern Detector Agent (runs every N signals, identifies cross-agent correlations) → Alert Engine (confidence-threshold triggers) → WebSocket Event Bus → FastAPI REST + WebSocket API → Streamlit Real-Time Dashboard (live signal feed, agent health, pattern alerts) → Docker Compose
Agent Roles	Company Monitor Agent: produces structured signals about company mentions and developments. Technology Signal Agent: tracks emerging technology patterns in a configured domain. Competitor Activity Agent: monitors competitor moves with structured classification (product launch, pricing change, partnership). Sentiment Agent: scores topic sentiment over time with

	trend detection. Regulatory Agent: flags regulatory developments relevant to configured industries. Pattern Detector Agent: runs across all signal types to surface correlations. Alert Engine: evaluates signals against user-defined rules and fires typed alerts. Each agent runs on a configurable schedule and produces a typed Signal object stored to SQLite.
Technical Skills	Scheduled agent execution, WebSocket real-time streaming, SQLite time-series design, pattern detection across agents, alert engineering, Streamlit real-time updates, Docker Compose multi-service, FastAPI WebSocket
Portfolio Deliverables	GitHub repo with system design README, Docker Compose deployment (starts full system with one command), Streamlit dashboard with live signal feed + pattern alerts + agent health panel, FastAPI REST API with full OpenAPI docs, WebSocket endpoint for dashboard, SQLite schema documented, 24-hour simulated run with recorded signal history, 3 configured intelligence topics demonstrated, architecture diagram, recorded 5-minute demo video
What Makes It Stand Out	Real-time multi-agent systems running on a schedule are categorically different from request-response agents. The pattern detection across agents finding correlations no single agent can see is the kind of emergent intelligence that makes a compelling interview story. A live dashboard that non-technical stakeholders understand is worth more than ten code samples.
Evaluation Criteria	At least 5 specialist agents run on schedule and produce typed Signal objects. Pattern Detector finds at least one real cross-agent pattern in a 10-minute test run. Alert Engine fires correctly on configured thresholds. WebSocket pushes signals to Streamlit dashboard in real time (< 2-second delay). Docker Compose starts cleanly. FastAPI endpoints documented. 24-hour simulated history committed to repo with analysis.

Week 5 Completion Checklist

Confirm every item below before Friday's standup. Projects that are not submitted via GitHub with a complete README by standup time will not receive full credit.

	Checklist Item
☐	All 5 daily learning sessions completed (Mon–Fri)
☐	Lab 5.1 Typed message bus implemented with 4 Pydantic schemas and validation tested
☐	Lab 5.2 Supervisor with failure recovery built and tested against 2 injected failure modes
☐	Lab 5.3 Consensus engine built with confidence-weighted voting and second-round logic
☐	Weekly Assessment all 6 questions answered in writing
☐	One Intermediate project chosen, built, and pushed to GitHub with full README
☐	One Production project chosen, built, and pushed to GitHub with system design README
☐	Both projects have architecture diagrams committed to the repo
☐	Production project README includes a clear local setup guide (one-command preferred)
☐	Demo rehearsed live demo showing multi-agent execution, decision logs, and failure handling